Foundations of Computing I
Summary

Simon Schuhmacher

UZH FS2026

Contents

1 Propositional Logic **1**
Basic Logical Equivalences . . . 1
Commutative Law . . . 1
Associative Law . . . 1
Distributive Law . . . 1
Identity Law . . . 1
Negation Law . . . 1
Double Negation Law . . . 1
Idempotent Law . . . 1
Universal Bound Law . . . 1
De Morgan's Law . . . 1
Absorption Law . . . 1
Negations of **t** and **c** . . . 1
XOR . . . 1
Implication (If-Then) . . . 1
Equivalence (Biconditional If and Only if) . . . 1

2 Digital Logic Circuits **1**
Disjunctive Normal Form (DNF) . . . 1
Conjunctive Normal Form (CNF) . . . 1
NOR Gate . . . 2
NOT Using NOR . . . 2
OR Using NOR . . . 2
AND Using NOR . . . 2
NAND Gate . . . 2
NOT Using NAND . . . 2
OR Using NAND . . . 2
AND Using NAND . . . 2

3 Induction and Recursion **2**
Proof by Mathematical Induction . . . 2
Proof by Strong Mathematical Induction . . . 2
Method of the Loop Invariant to Prove Algorithm Correctness . . . 2

4 Functions and Convolution **3**
Laws of Exponents . . . 3
Laws of Logarithms . . . 3
1D Convolution . . . 3
Properties of Convolution . . . 3
Why We Flip the Function in the Definition of Convolution . . . 3
2D Convolution . . . 3

1 Propositional Logic

Basic Logical Equivalences

Commutative Law

- $p \wedge q \equiv q \wedge p$
- $p \vee q \equiv q \vee p$

Associative Law

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
- $(p \vee q) \vee r \equiv p \vee (q \vee r)$

Distributive Law

- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Identity Law

- $p \wedge \mathbf{t} \equiv p$
- $p \vee \mathbf{c} \equiv p$

Negation Law

- $p \vee \sim p \equiv \mathbf{t}$
- $p \wedge \sim p \equiv \mathbf{c}$

Double Negation Law

- $\sim(\sim p) \equiv p$

Idempotent Law

- $p \wedge p \equiv p$
- $p \vee p \equiv p$

Universal Bound Law

- $p \vee \mathbf{t} \equiv \mathbf{t}$
- $p \wedge \mathbf{c} \equiv \mathbf{c}$

De Morgan's Law

- $\sim(p \wedge q) \equiv \sim p \vee \sim q$
- $\sim(p \vee q) \equiv \sim \wedge \sim q$

Absorption Law

- $p \vee (p \wedge q) \equiv p$
- $p \wedge (p \vee q) \equiv p$

Negations of t and c

- $\sim\mathbf{t} \equiv \mathbf{c}$
- $\sim\mathbf{c} \equiv \mathbf{t}$

XOR

- $p \oplus q \equiv (p \vee q) \wedge \sim(p \wedge q)$
- $p \oplus q \equiv (a \wedge \sim b) \vee (\sim a \wedge b)$

Implication (If-Then)

- $p \to q \equiv \sim p \vee q$

Equivalence (Biconditional If and Only if)

- $p \leftrightarrow q \equiv (p \to q) \wedge (q \to p)$

2 Digital Logic Circuits

Disjunctive Normal Form (DNF)

"OR of ANDs"

From truth table to DNF

1. Identify the rows whose output is 1
2. AND together all variables in the same row (negate the variable when 0)
3. OR together the 1-rows

Conjunctive Normal Form (CNF)

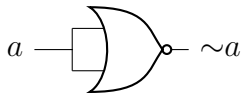
"AND of ORs"

From truth table to CNF

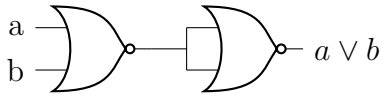
1. Identify the rows whose output is 0
2. OR together all variables in the same row (negate the variable when 1)
3. AND together the 0-rows

NOR Gate

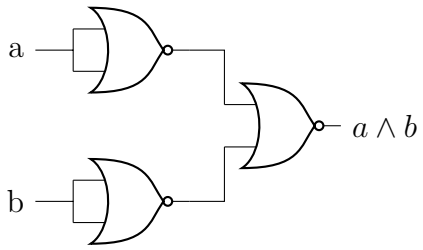
NOT Using NOR



OR Using NOR

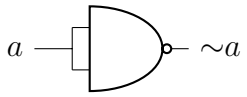


AND Using NOR

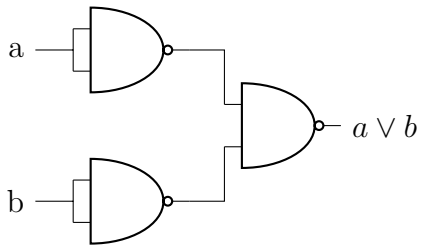


NAND Gate

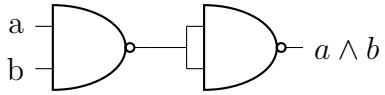
NOT Using NAND



OR Using NAND



AND Using NAND



3 Induction and Recursion

Proof by Mathematical Induction

To prove that a property $P(n)$ is **true** for every integer $n \geq b$:

1. **Base Case**: Show that $P(b)$ is true
2. **Inductive Hypothesis**: Assume $P(k)$ true for any $k \geq b$
3. **Inductive Step**: **Show** that $P(k) \to P(k+1)$ **is true**
4. **Conclusion**: If $P(b)$ is true and $P(k) \to P(k+1)$ is true, conclude that $P(n)$ is true for all $n \geq b$

Proof by Strong Mathematical Induction

- Prove recursive problems with several base cases (e.g., recurrence relations with order larger than 1)
- Makes a stronger hypothesis than the principle of Mathematical Induction:
  - The **base step** may contain proofs for **several initial values**, rather than just the base case
  - In the inductive step, the trust of $P(k)$ is assumed not just for an arbitrary $k \geq b$ but for **all values through** k

Let b, b be two integers with $a \leq b$. To prove that a property $P(n)$ is **true** for every integer $n \geq a$:

1. **Base Case**: Show that $P(n)$ is true for all base cases from a to b, that is: $P(a) \wedge P(a+1) \wedge \cdots \wedge P(b)$
2. **Inductive Hypothesis**: For any $kgeqb$, assume $P(i)$ **is true for** $i = a, \ldots, b, \ldots, k$
3. **Show** that $P(k+1)$ **is true**
4. **Conclusion**: If 1 and 2 are verified, conclude that $P(n)$ is true for all $n \geq a$

Method of the Loop Invariant to Prove Algorithm Correctness

For every $n \geq 0$, let $P(n)$ be the loop invariant for a loop. **If the following four properties are true, then the loop is correct** with respect to its pre- and post-conditions:

1. **Basic property**: the loop invariant is true before the loop starts, i.e., $P(0)$ is true before the loop starts. ($P(0)$ is $P(n)$ when $n = 0$). → "The loop invariant is true before the loop starts (i.e., base case is true)"
2. **Inductive property**: for any integer $k \geq 0$, if **both the guard G and the loop invariant** $P(k)$ are true before an iteration of the loop, then $P(k+1)$ is true after iteration. → "It remains true after every iteration (i.e., $P(k) \to P(k+1)$)"
3. **Eventual Falsity of the Guard**: after a finite number of iterations of the loop, the guard G becomes false. → "The loop eventually stops (i.e., that the loop condition eventually turns false)"
4. **Correctness of the post-condition**: if N is the number of iterations after which G is false and $P(n)$ is true, then the post-condition is satisfied. → "At the point when the loop ends, you need to verify that the **loop invariant** is true"

4 Functions and Convolution

Laws of Exponents

If b and c are any positive real numbers and u and v are any real numbers, the following laws of exponents hold true:

- $b^u b^v = b^{u+v}$
- $(b^u)^v = b^{uv}$
- $\frac{b^u}{b^v} = b^{u-v}$
- $(bc)^u = b^u c^u$

Laws of Logarithms

For any positive real numbers b, c and x with $b \neq 1$ and $c \neq 1$:

- $\log_b(xy) = \log_b x + \log_b y$
- $\log_b(\frac{x}{y}) = \log_b x - \log_b y$
- $\log_b(x^a) = a \log_b x$
- $\log_c x = \frac{\log_b x}{\log_b c}$

1D Convolution

$f(n)$, $g(n)$, $y(n)$: $\mathbb{Z} \to \mathbb{R} \quad \forall n \in \mathbb{Z}$

1D convolution between f and g, denoted with $y(n) = (f * g)(n) = \sum_{k=-\infty}^{+\infty} f(k)g(n-k)$

- f: input function
- g: kernel or filter

Properties of Convolution

- **Commutative Law**: $f * g = g * f$
- **Associative Law**: $(f * g) * h = f * (g * h)$
- **Associative Law with Scalar Multiplication**: $(\lambda f) * g = f * (\lambda g) = \lambda(f * g)$
- **Distributive Law**: $f * (g + h) = (f * g) + (f * h)$
- **Convolution with a Dirac**: $f * \delta = f$

Why We Flip the Function in the Definition of Convolution

$(f * g)(n) = (g * f)(n) \Rightarrow$
$\sum_{k=-\infty}^{+\infty} f(k)g(n-k) = \sum_{k=-\infty}^{+\infty} g(k)f(n-k)$

- Without flipping, convolution would not be commutative

2D Convolution

Let F, G, Y be **functions** defined on the infinite set $\mathbb{Z}^2$: $F(i,j)$, $G(i,j)$, $Y(i,j)$: $\mathbb{Z}^2 \to \mathbb{R} \quad \forall (i,j) \in \mathbb{Z}^2$

We define the 2D convolution between f and g: $Y(i,j) = (F * G)(i,j) =$
$\sum_{u=-\infty}^{+\infty} \sum_{v=-\infty}^{+\infty} F(u,v)G(i-u, j-v)$

- f: input image
- g: kernel or filter